Factory ERP

Development Summary

Everything designed and built to date: the architecture, the production-grade foundation, GST invoicing, the sales-to-cash receivables loop, and the sales correction path (invoice cancellation and credit notes).

Working prototype | Synthetic demo data | Specimen mode (pending CA validation) | All checks green

229	36	37	≈21,100
automated tests, all passing	browser end-to-end checks	API endpoints	lines of hand-written code (178 files)

Confidential · prototype for internal review

Contents

1 Executive summary
2 Project journey
3 Architecture as built
4 Technology stack
5 Repository structure
6 Platform layer
7 Identity, sessions & permissions
8 Database & data integrity
9 Money & the GST engine
10 Sales invoicing
11 Sales-to-cash: receipts & receivables
12 Sales corrections: cancellation & credit notes
13 Dashboard & KPIs
14 API reference
15 Web application
16 Demo data
17 Testing & verification
18 Defects found & fixed
19 Running, building & CI
20 Documentation produced
21 Limitations & open decisions
22 Recommended next steps

1 Executive summary

Factory ERP (renamed from “Factory OS” on 7 October 2026, so it is not mistaken for an operating system) is an AI-native ERP for Indian MSME factories. It runs on a local factory server, works over the factory LAN without internet, and is reachable remotely through one secure address. It is being built first as a surprise prototype for one real factory. Factory discovery is deferred, so it runs on clearly labelled synthetic data.

In this phase the project went from an empty folder to a working, tested application.

- **A production-grade foundation.**
  - A TypeScript monorepo with PostgreSQL 18, forward-only SQL migrations and generated database types.
  - A typed command and query bus. Every change is validated, authorised, transactional, idempotent and audited.
  - Role-based access with eight roles, append-only audit and ledger tables, a transactional outbox, gapless document numbering, structured logging and a CI pipeline.
- **GST invoicing.** Drafts priced by a pure, deterministic GST engine on the server. Issuing assigns a gapless number per financial year and freezes the invoice in the database itself. Each issued invoice gets a PDF with a *SPECIMEN* watermark until a CA has validated the calculations.
- **Sales-to-cash.** Customer receipts, bill-wise allocation (full, partial, one receipt across several invoices) and an explicit overpayment rule. Outstanding balances, payment status and ageing are always *derived*, never stored.
- **The sales correction path.**
  - **Invoice cancellation:** nothing is deleted; who, when and why are recorded; a reversing ledger entry corrects the balance.
  - **Credit notes:** a separate, numbered, immutable document, with partial and full credits against one invoice.
  - The database and the application both guarantee that no invoice is ever over-settled or pushed into negative outstanding.
- **A dense, keyboard-first ERP web app.** Served by the same server, so it has one origin for LAN and remote use. It is mobile-friendly and has a Ctrl/⌘+K command palette.

TESTS	BROWSER E2E	STATIC CHECKS
229 / 229	**36 / 36**	**Clean**
unit, property-based & real-PostgreSQL integration	smoke (11) · receivables (10) · corrections (15)	format · lint · strict typecheck · schema drift

Status, honestly stated

- This is a verified prototype, not yet fit for real tax documents.
- GST rates are demo values marked *unverified*, and invoice and credit-note PDFs are watermarked SPECIMEN.
- Rounding policy, credit-note rules, cancellation rules and the treatment of advances need sign-off from your chartered accountant.
- Refunds and receipt reversals are deliberately not built until those accounting decisions are made.

2 Project journey

Stage	What happened	Output
Repository inspection 5 Oct 2026	The folder was empty: no code and no git. Development started from scratch.	Assessment
Master architecture v0.1	Full proposal, sections A–S: stack, topology, domain model, database, modules and API, AI architecture, local inference, auth and RBAC, offline mode, documents, invoicing, accounting, testing, deployment, security, phases, risks and decisions.	`docs/ARCHITECTURE.md`
Architecture Review, Round 2	Factory V1 separated from SaaS-later; topology revised; stack made version-agnostic; regulatory claims turned into a *[VERIFY R-nn]* register; CA validation pipeline, outage matrix, hardware-adaptive AI and a hostile review added; decisions R2-D1…D7 approved.	Architecture v0.2
Discovery pack	Printable factory discovery pack, a questionnaire of about 150 questions, and a data checklist. **Deferred** because the project is a surprise prototype; kept for future validation.	`docs/discovery/` (+ PDF)
Phase 0 + invoicing slice 5 Oct 2026	Monorepo, tooling, CI and Docker Postgres; platform (config, logging, migrations, buses, auth, RBAC, audit, idempotency, outbox); authentication → company → parties → items → draft → GST → issue → PDF → audit → dashboard; web app and e2e.	125 tests, e2e 11/11
Sales-to-cash loop 7 Oct 2026	Receipts, bill-wise allocation and the overpayment rule; derived settlement, ageing and the receivables summary; receipt and receivables UI; demo payments; concurrency proofs; a second e2e journey.	172 tests, e2e 21/21
Sales correction path + rename 7 Oct 2026	• Renamed to **Factory ERP** everywhere it matters (UI, PDFs, docs, packages, database and Docker names). • Invoice cancellation and credit notes (migration 0006, ADR 0002), with ledger, receivables, dashboard, PDF, UI and API support. • A self-contained e2e runner on a throwaway database; five new correction journeys.	229 tests, e2e 36/36

Key decisions in force

ID	Decision
S1	Build our own core rather than wrapping Odoo or ERPNext. Those products are studied for domain concepts only.
S3	The factory server is the **single authoritative writer**. There is no multi-master sync.
R2-D1	The factory server serves the web app itself; one origin serves both LAN and remote access.
R2-D3	TypeScript is the primary language. Python is reserved for specialist OCR, vision and optimisation sidecars.
R2-D4	V1 accounting covers immutable vouchers, customer and supplier balances, and (later) Tally export. Tally stays the official books.
R2-D6	The remote-access provider is not yet locked (Cloudflare Tunnel vs. alternatives).

ADR 0002	• An issued invoice is cancelled only if nothing has settled it (no payments, no credit notes). • A credit note is its own document against exactly one invoice; it can never make outstanding negative. • Invoice cancellation is a CONFIRM-tier action: the AI may propose it, only a human can execute it.
Build	Factory discovery is deferred. Everything runs on synthetic demo data; every factory assumption is provisional.

3 Architecture as built

Core principle: database → business logic → APIs → AI tools

Every mutation is a **typed command** registered once and executed by the command bus. HTTP routes are thin adapters over it, and the future AI copilot will use the very same commands. There is **no privileged path**: the AI can never do something the API cannot, and it never skips a check.

validate (Zod) → authenticate → authorise (RBAC + AI tier) → idempotency → transaction → domain rules → persist → audit → outbox → response + log

Amber steps run inside one PostgreSQL transaction. If any step fails, everything rolls back, including the audit row.

Principles enforced in code

- **No floats for money.** Money values are exact decimal strings end to end. Lint rules ban `parseFloat` and `Math.round` in the financial packages.
- **Deterministic, server-authoritative calculations.** The browser shows previews only; the server recomputes on save and again on issue.
- **Corrections are new records, never edits.** Audit events, the party ledger, receipts, allocations, stock movements and documents are append-only; issued invoices and credit notes are immutable. Database triggers enforce this.
- **Derived balances.** Invoice outstanding (grand total – payments – credit notes), payment status, unallocated receipt money and stock on hand are always computed.
- **Regulatory values are data, not constants.** GST rates are effective-dated rows with a verification status; number-series limits are configurable.
- **The AI copilot is governed by risk tiers:** DRAFT the AI may execute (prepare drafts) · CONFIRM a human must click · FORBIDDEN never exposed · queries are READ.

Runtime topology (Factory V1)

Browser (LAN or remote)
React PWA shell served by the core. One origin. Cookie session (HttpOnly, SameSite=Lax).

Core (Node 24 + Fastify)
REST API `/api/v1` and static web app · command and query bus, auth, RBAC, audit · outbox worker rendering invoice and credit-note PDFs in headless Chrome.

PostgreSQL 18
Single source of truth: 10 schemas, guard triggers, deferred invariant checks and derived views. Files go to a content-addressed blob store.

Module layering

Layer	Modules	Rule
Platform	commands, queries, audit, idempotency, outbox, numbering, blob store, PDF, clock, ids, errors	Shared infrastructure; knows no business module
Masters	company, parties, catalog (items, HSN, GST rates), identity	Used by everything above
Documents & ledgers	sales (invoices, cancellation, credit notes, settlement read model), finance (party ledger)	Documents post to the ledger in the same transaction
Settlement	receivables (receipts, allocation, ageing)	Every settlement write — payment, credit note, cancellation — takes one sales-owned invoice lock

Composition	analytics (dashboard KPI registry), audit history	Reads only and composes module insights

4 Technology stack

Versions were verified against the npm registry and peer-dependency ranges, then recorded in `docs/adr/0001-foundation-stack-and-versions.md` and pinned exactly in `pnpm-lock.yaml`.

Concern	Choice (version)	Why
Runtime	Node.js 24 LTS (≥ 24.7)	Active LTS. Built-in `crypto.argon2`, so no native password add-on is needed.
Package manager	pnpm 11.18 (via corepack)	Workspaces. Dependency build scripts are blocked by default, with only esbuild approved.
Language	TypeScript 6.0.3 (strict)	TS 7 exists, but typescript-eslint supports only < 6.1, and type-aware linting was kept.
API	Fastify 5.12, @fastify/cookie, static, rate-limit	Fast and explicit. Rate limiting is applied to login only.
Contracts	Zod 4.6	One schema drives API validation, web types and (later) AI tool definitions.
Database	PostgreSQL 18.6 (Docker, localhost-bound)	Transactional DDL, rich constraints, exclusion constraints, deferrable constraint triggers.
DB access	Kysely 0.29 + kysely-codegen 0.20	Typed SQL with types generated from the real schema. CI verifies there is no drift.
Decimal maths	decimal.js 10.6 → Money / Qty / Rate / Percent	Exact arithmetic with explicit rounding modes.
Web	React 19.3, Vite 8.3, TanStack Router 1.170 / Query 5.104, Tailwind 4.3	Single-page app built as a PWA shell, served by the core.
PDF	Headless Chrome/Chromium via playwright-core 1.63	HTML/CSS templates with correct ₹ and Indic shaping. No bundled browser.
Logging	pino 10 (JSON), with redaction of cookies and passwords	Structured logs carrying a request ID.
Testing	Vitest 5, fast-check 4, playwright-core	Unit, property-based, real-Postgres integration and browser e2e tests.
Quality	ESLint 10 + typescript-eslint 8.71 (type-aware), Prettier 3.9	Floating promises, `any` and float money are all banned.

> **Deliberately not used**
> Redis, Kafka, Kubernetes, microservices, GraphQL, ORM magic and WebSocket (polling in V1). pg-boss is not used either: a transactional outbox with `FOR UPDATE SKIP LOCKED` covers V1.

5 Repository structure

```
factory-erp/
├─ apps/core/          Fastify API · platform · modules · seed · tests
├─ apps/web/           React + Vite web app · e2e journeys + self-contained runner
├─ packages/money/     Money/Qty/Rate/Percent, Indian formatting, amount in words
├─ packages/gst/       Pure GST invoice engine + Indian financial year
├─ packages/contracts/ Zod schemas, DTO types, permission codes (shared API ↔ web ↔ AI)
├─ db/migrations/      0001 … 0006 plain SQL — the schema is the source of truth
├─ docs/               Architecture, ADR 0001–0002, discovery pack (deferred), this summary
├─ compose.yaml        PostgreSQL 18 for development (project "factory-erp")
└─ .github/workflows/  CI: format · lint · typecheck · migrate · codegen verify · test · build · e2e
```

Area	Files	Lines
apps/core/src (API, platform, modules, seed, integration tests)	91	10,595
apps/core/test (test DB harness, fixtures, business flows)	4	472
apps/web/src (UI)	45	5,823
apps/web/e2e (browser journeys and runner)	5	978
packages/money · gst · contracts	27	2,134
db/migrations	6	1,068
Total (excluding generated DB types)	**178**	**21,070**

6 Platform layer

Component	What it does
Command bus	• Each command is registered once, with a name, description, Zod input, permission, AI tier and handler. • The handler *must* return at least one audit record; the type system enforces this. • Translates database constraint errors into client-safe errors and logs every execution.
Query bus	Runs the same validation and authorisation for reads, and gives handlers `can(permission)` to include optional fields only for authorised users.
Idempotency	The `Idempotency-Key` header is required on every command request. Repeats replay the stored response, concurrent duplicates execute once, and reusing a key for a different request is rejected (422).
Audit	`audit.events` is written in the same transaction as the change. Each event records actor, actor kind (user / ai_agent / system / integration), AI session, request ID, command, entity and before/after/metadata. Append-only, enforced by a trigger.
Transactional outbox	Events are written with the business change. A poller claims them with a lease (`FOR UPDATE SKIP LOCKED`), processes each in its own transaction, retries with exponential backoff, and marks them FAILED after 8 attempts. Topics today: `sales.invoice.issued` and `sales.credit_note.issued` (both render PDFs).
Document numbering	• One series per company × document type × financial year, locked `FOR UPDATE` : gapless, consecutive, and a rollback also rolls back the increment. • A document date cannot precede the last numbered date in its series. Maximum length is configurable data *[VERIFY R-02]*. • Shared by invoices (`INV/26-27/00001`), receipts (`RCPT/26-27/00001`) and credit notes (`CN/26-27/00001`).
Blob store	Content-addressed SHA-256 local file storage, using write-then-rename. Document downloads re-verify the hash.
PDF renderer	One shared headless Chrome instance; JavaScript disabled and all network requests blocked. The server's graceful shutdown finishes in-flight PDF jobs before closing the browser.
Config & logging	The environment is validated by Zod at startup without echoing secrets; production requires secure cookies; JSON logs carry request IDs with secrets redacted.
Errors	RFC 9457 `application/problem+json` with stable codes and a rule name. Database SQLSTATEs `F0001` (append-only), `F0002` (immutable document), `F0003` (invalid allocation) and `F0004` (invalid credit note) are mapped to client-safe errors.
HTTP hardening	CSP, X-Frame-Options DENY, nosniff, Referrer-Policy; the `text/plain` body parser is removed (CSRF hardening); JSON 404s for unknown API routes.
Clock & IDs	Business dates are always in IST, with day boundaries and financial-year rollover tested. UUIDv7 IDs are generated in the application.

7 Identity, sessions & permissions

- **Passwords:** argon2id using Node's built-in crypto, stored as PHC strings. Unknown usernames still run one hash check, so timing doesn't reveal which usernames exist.
- **Sessions:** random 256-bit tokens, of which only the SHA-256 is stored. Cookies are `HttpOnly` and `SameSite=Lax` with a 12 h TTL. Login and logout are audited; login is rate-limited.
- **Permissions are code; roles are data.** Sixteen permission codes live in `packages/contracts`. A reviewed snapshot test makes any change to who-can-do-what visible.
- **The UI hides what a role can't do,** but the server enforces every permission independently.

Role / permission matrix (reviewed snapshot)

Permission	Owner	Admin	Accountant	Prod. Mgr	Purchase Mgr	HR	Supervisor	Worker
company.read	✓	✓	✓	✓	✓	✓	✓	✓
party.read · party.create	✓	✓	✓		✓			
item.read	✓	✓	✓	✓	✓		✓	
item.create	✓	✓			✓			
sales.invoice.read · draft · issue	✓	✓	✓					
sales.invoice.cancel new	✓	✓	✓					
sales.credit_note.draft · issue new	✓	✓	✓					
finance.receivables.read · finance.receipt.record	✓	✓	✓					
inventory.stock.read	✓	✓	✓	✓	✓		✓	
document.read · audit.read	✓	✓	✓					

AI copilot tiers already enforced

The copilot itself is not built yet, but its guard rails are already in the bus. An actor with `actorKind: 'ai_agent'` may execute only DRAFT-tier commands, with exactly the user's permissions, and is audited as *ai_agent* together with its session ID. Integration tests prove that the AI can prepare credit-note drafts but can never issue, delete or cancel anything.

Command	Permission	AI tier
parties.create · items.create	party.create · item.create	CONFIRM
sales.invoice.createDraft · updateDraft	sales.invoice.draft	DRAFT
sales.invoice.deleteDraft · sales.invoice.issue	sales.invoice.draft · issue	CONFIRM
sales.invoice.cancel	sales.invoice.cancel	CONFIRM
sales.creditNote.createDraft · updateDraft	sales.credit_note.draft	DRAFT
sales.creditNote.deleteDraft · sales.creditNote.issue	sales.credit_note.draft · issue	CONFIRM
finance.receipt.record · finance.receipt.allocate	finance.receipt.record	CONFIRM

8 Database & data integrity

PostgreSQL 18 holds 10 application schemas: `platform, identity, audit, system, party, catalog, sales, finance, docs, inventory`. Migrations are forward-only plain SQL; each runs in its own transaction, is checksummed, and runs under an advisory lock. Editing an applied migration is refused. Types are generated from the live schema.

Migration	Adds
`0001_platform_identity_audit`	Guard functions; 37 GST state codes *[VERIFY R-09]*; companies; users, roles, sessions; **audit.events** (append-only); idempotency keys, outbox events, number series.
`0002_parties_catalog`	Parties (customers and suppliers); units *[VERIFY R-14]*; HSN codes; **effective-dated GST rates** with an exclusion constraint forbidding overlaps; items.
`0003_sales_finance_documents`	Documents (append-only); invoices with status and immutability guards; invoice lines; **party_ledger_entries** (append-only).
`0004_inventory`	Warehouses, append-only stock movements and the stock-on-hand view.
`0005_receipts_allocations`	Receipts and allocations (append-only); a deferred allocation check at COMMIT (FO003); views **invoice_balances** and **receipt_balances**.
`0006_sales_corrections` new	• Write-once cancellation details on invoices; the cancellation ledger entry type; a credit-note prefix. • **credit_notes** and **credit_note_lines** with a composite foreign key to (invoice, customer, company) and per-line references to invoice lines. • Immutability guards, a deferred credit-note check (FO004), credit-aware allocation check and balances view.

Defence in depth: the database guards the invariants itself

Invariant	Mechanism
Financial history is never edited or deleted	BEFORE UPDATE/DELETE/TRUNCATE triggers (FO001) on audit, ledger, receipts, allocations, stock and documents
Issued invoices and credit notes are immutable	Row-diff guard triggers (FO002) plus line guards and delete guards
Cancellation is recorded once, by the right transition only	CHECK (who/when/why mandatory exactly when CANCELLED) + guard trigger (no edits, no un-cancel, no draft cancellation, no cancelling a paid or credited invoice)
Σ payments + Σ credit notes ≤ invoice total, even through raw SQL	Deferred constraint triggers at commit (FO003 for allocations, FO004 for credit notes)
A credit note credits only its own customer's invoice, in its own company	Composite foreign key `(invoice_id, customer_id, company_id)`
Gapless, unique numbering	Series row lock plus unique constraints on invoice, receipt and credit-note numbers
No overlapping tax rates for an HSN	GiST exclusion constraint on date ranges

9 Money & the GST engine

packages/money

- **Value types:** `Money` (≤ 2 dp), `Qty` (≤ 3 dp), `Rate` (≤ 4 dp), `Percent` (0–100). Built from strings only; rounding only through explicitly named modes.
- **Indian formatting** in lakh/crore grouping (`₹12,34,567.80`), verified against `Intl` en-IN by a property test; **amount in words** in the Indian system.

packages/gst — pure, deterministic, versioned (engine v1.0.0)

- **Input:** supplier state, place of supply, lines and policy. **Output:** per-line taxable amount and CGST/SGST or IGST, rate-wise summary, totals, round-off and amount in words.
- **Policy** (every option is a **CA validation point**): line-level HALF_UP rounding to the paisa; grand total rounded to the nearest rupee or not at all.
- **Reused unchanged by credit notes:** a credit note is priced by the same engine with the original invoice's supply type, line GST rates, discounts and stored rounding policy, so a full credit reproduces the invoice total exactly.
- **Versioning:** issued documents store the engine version and every computed value, and are never recomputed.

Worked example: invoice INV/26-27/00006 (demo)

Line	Qty × rate	Taxable	GST	CGST	SGST	Total
Bracket Assembly BA-100	400 × ₹245.00	₹98,000.00	18%	₹8,820.00	₹8,820.00	₹1,15,640.00
Corrugated Shipping Carton	20 × ₹38.50	₹770.00	5%	₹19.25	₹19.25	₹808.50
Totals		**₹98,770.00**		**₹8,839.25**	**₹8,839.25**	**₹1,16,448.50**
Round off (nearest rupee, HALF_UP)						+₹0.50
Grand total — "Rupees One Lakh Sixteen Thousand Four Hundred Forty Nine Only"						**₹1,16,449.00**

> **A real finding for the CA**
> Rounding CGST and SGST separately per line can differ by a paisa from rounding the total tax once (9% of ₹85.08 = ₹7.6572 → ₹7.66 twice = ₹15.32, not ₹15.31). The same effect applies when one invoice line is credited in several parts. The engine exposes this as an explicit, tested policy for your chartered accountant to confirm.

Tests

14 hand-worked cases, 6 financial-year cases, and property-based tests that run **400 random invoices** per run, checking that totals equal the sum of lines, CGST equals SGST, the supply type decides which taxes apply, |round-off| ≤ ₹0.50, the rate summary reconciles, and the engine is deterministic.

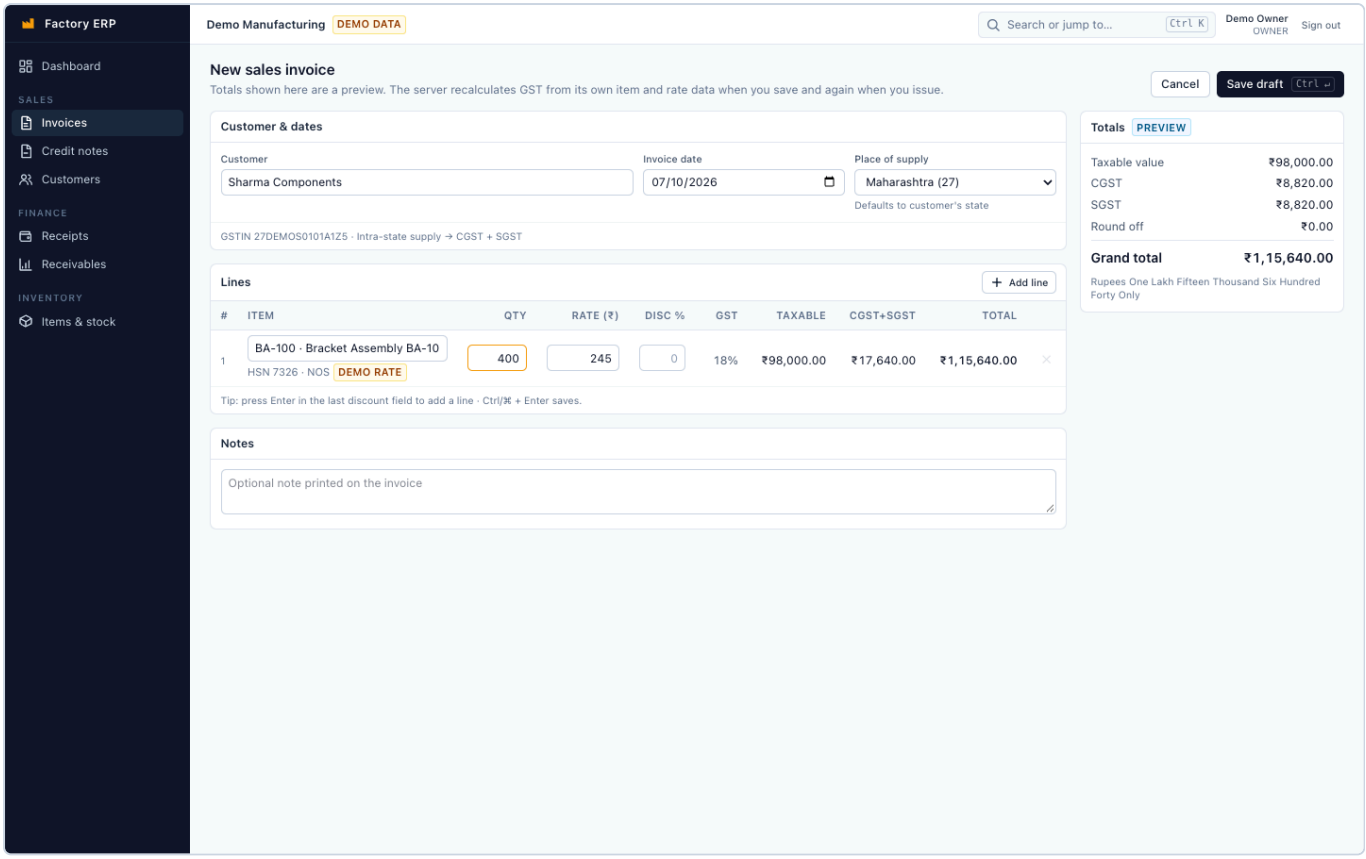
10 Sales invoicing

Lifecycle

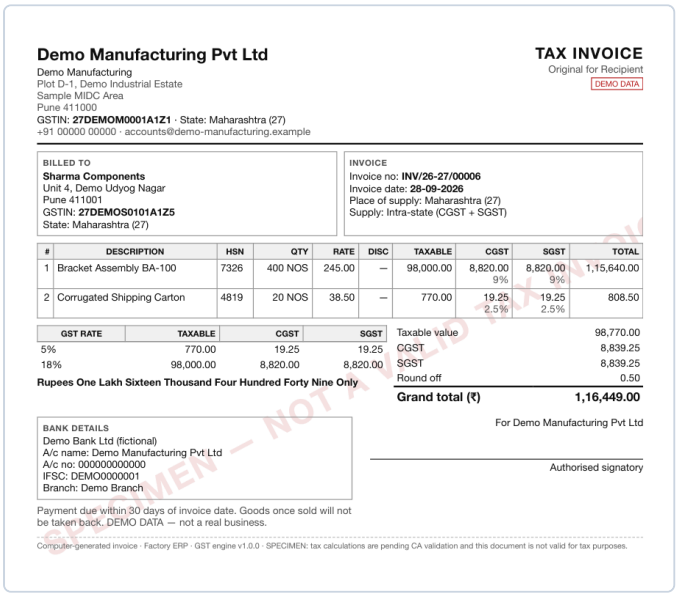
DRAFT → ISSUED → CANCELLED (see chapter 12). Payment status is derived separately: UNPAID / PART-PAID / PAID / CREDITED (cleared by credit notes rather than money).

Issue: one transaction, ten steps

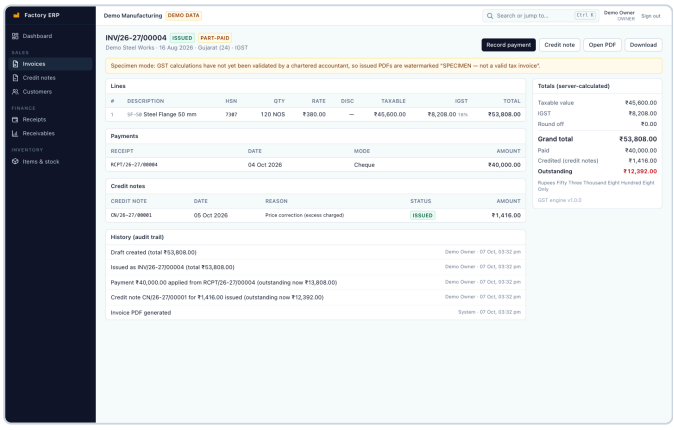
1. Lock the draft and check its version; reject a future-dated invoice.
2. Load seller and customer; recalculate authoritatively from current master data and the rates in force on the invoice date.
3. Allocate the next gapless number (date must not precede the series' last date).
4. Snapshot the seller and the customer; mark ISSUED — from here the database makes it immutable.
5. Post a debit to the customer's balance; write the audit record; add an outbox event that renders the PDF.



Invoice editor: keyboard-first line grid, demo-rate badge, live GST preview (the server recalculates on save and issue)



Generated invoice PDF: SPECIMEN watermark until finance readiness is PRODUCTION; DEMO DATA badge



Invoice INV/26-27/00004: part-paid, a ₹40,000 payment and a ₹1,416 credit note; Paid / Credited / Outstanding derived; full audit history

11 Sales-to-cash: receipts & receivables

Customer → issued invoice → receivable → receipt → allocation to one or more invoices → outstanding updates deterministically → ageing and reporting.

Domain rules

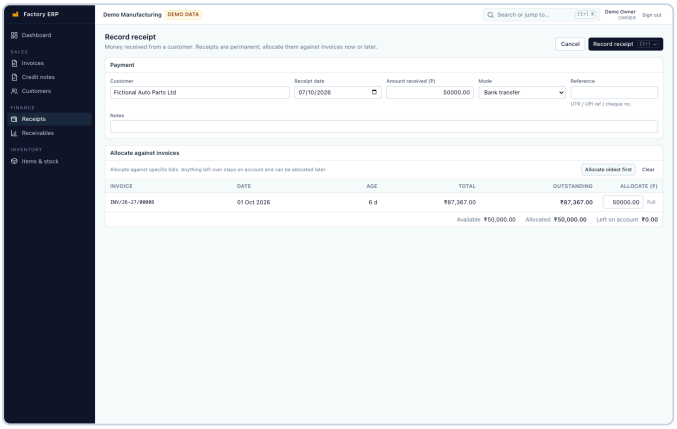
1. Money can be allocated only to **ISSUED** invoices of the **receipt's customer** (never to drafts or cancelled invoices).

2. An allocation may not exceed the invoice's **outstanding** amount: grand total – Σ allocations – Σ issued credit notes, always derived.
3. Σ allocations of a receipt may not exceed the receipt amount; requests are validated whole.

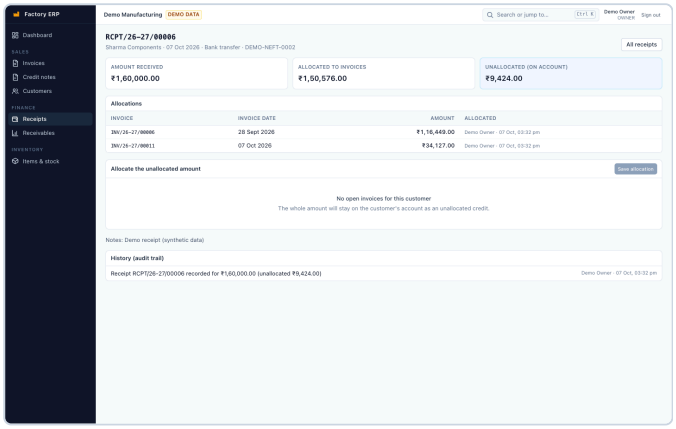
> **Explicit overpayment rule**
> A receipt may be larger than everything the customer owes. The excess stays as an **unallocated (on-account) credit** that lowers the customer's balance (which may go negative) and can be allocated to future invoices. Refunds are not modelled yet; the tax treatment of advances is flagged for the CA.

Ageing & reconciliation

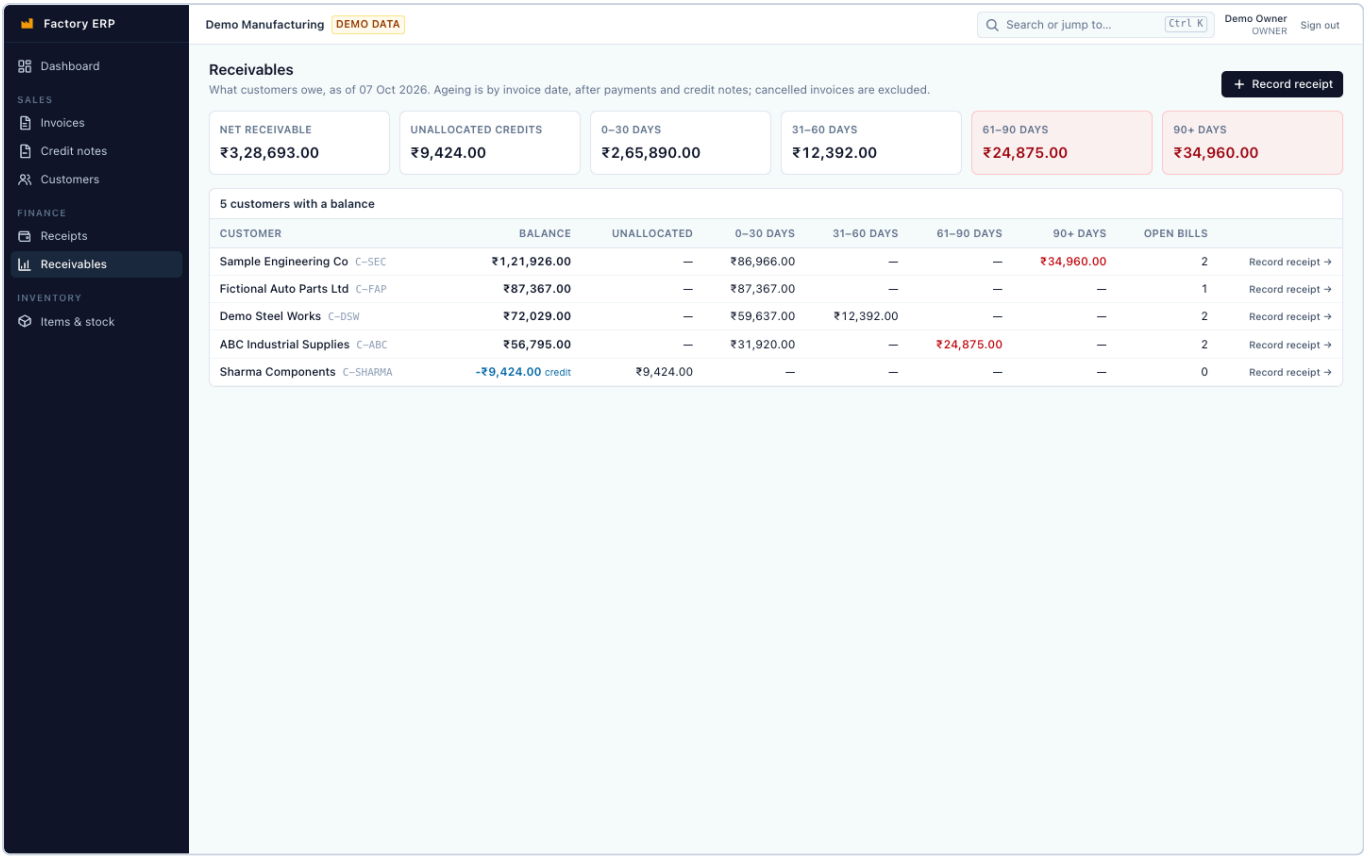
- **Buckets** by days since invoice date: 0–30, 31–60, 61–90 and 90+, on outstanding after payments and credit notes.
- **Summary per customer:** net invoiced (after cancellations), credited, received, balance, outstanding, unallocated, open bills and ageing.
- **Invariant, tested for every customer and the totals:** `balance = invoiced − credited − received = Σ outstanding − Σ unallocated`, and it equals the customer ledger.



Record receipt: bill-wise allocation grid with "Allocate oldest first" and live "left on account"



Receipt RCPT/26-27/00006: one receipt paying two invoices, ₹9,424 on account



Receivables: net balance, unallocated credits, ageing buckets (61+ days in red), per-customer table; Sharma Components in credit

12 Sales corrections: cancellation & credit notes NEW

Issued documents are never edited or deleted. A mistake or a return is corrected by a **new record**, so the immutable invoice, the credit notes, the ledger entries and the audit trail together always show exactly what happened. Full design: `docs/adr/0002-sales-correction-path.md`.

Invoice cancellation

- **Explicit command** (`sales.invoice.cancel`) with optimistic concurrency and a mandatory reason.
- **Nothing is lost:** the invoice keeps its number, values, snapshots and PDF. Who, when and why are written once and can never change — enforced by a CHECK constraint and a trigger.
- **Balance corrected by a new entry:** an `INVOICE_CANCELLATION` credit equal to the original debit; a unique key makes a second reversal impossible.
- **Only what nothing has settled:** invoices with payments or credit notes cannot be cancelled. That would need a refund or un-allocation — separate accounting decisions, deliberately not invented.
- **Afterwards:** no payment can be applied to it, no credit note issued against it, and it drops out of receivables, ageing and sales KPIs.

Credit notes

- **A separate document, not a negative invoice:** DRAFT → ISSUED, then immutable; its own gapless series `CN/<FY>/<nnnnn>` through the existing numbering.
- **Against exactly one issued invoice** of the same customer and company. Each line credits one invoice line as quantity × rate with that line's discount and GST rate: the invoiced rate for returns, the reduction per unit for price corrections.
- **Same engine, same tax position:** engine version, all values and seller/customer snapshots are stored at issue; a full credit reproduces the invoice exactly.

- **Issue is one transaction:** lock the draft → lock the invoice → re-check every rule on the locked state → number → snapshots → ISSUED → customer-ledger credit → audit on the credit note *and* the invoice → outbox event for the PDF.

Caps — nothing can be over-credited

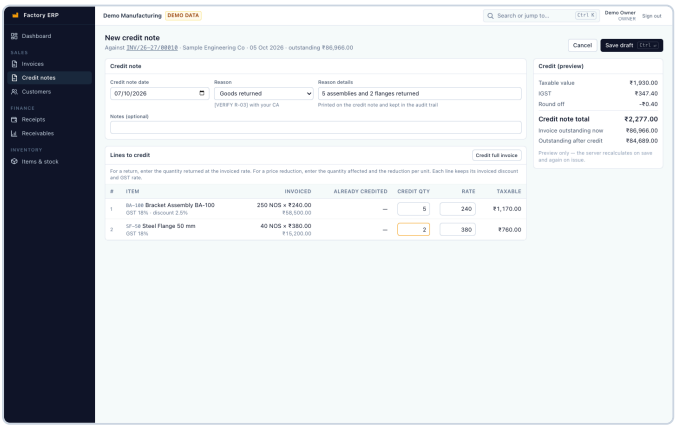
Rule	Code
Credit quantity ≤ invoiced quantity, credit rate ≤ invoiced rate (per line)	`EXCEEDS_LINE_QUANTITY` · `EXCEEDS_LINE_RATE`
Σ credited taxable value across all issued credit notes ≤ the line's invoiced taxable value	`EXCEEDS_LINE_CREDITABLE`
Credit note total ≤ invoice outstanding (total – payments – earlier credits): outstanding never goes negative	`EXCEEDS_INVOICE_OUTSTANDING`
Wrong customer · wrong company · draft or cancelled invoice	`CUSTOMER_MISMATCH` · not found · `INVOICE_NOT_ISSUED`
Not before the invoice date; not in the future at issue; more than ₹0	`CREDIT_NOTE_BEFORE_INVOICE` · `…_DATE_IN_FUTURE` · `CREDIT_NOTE_ZERO`

Concurrency (and proof)

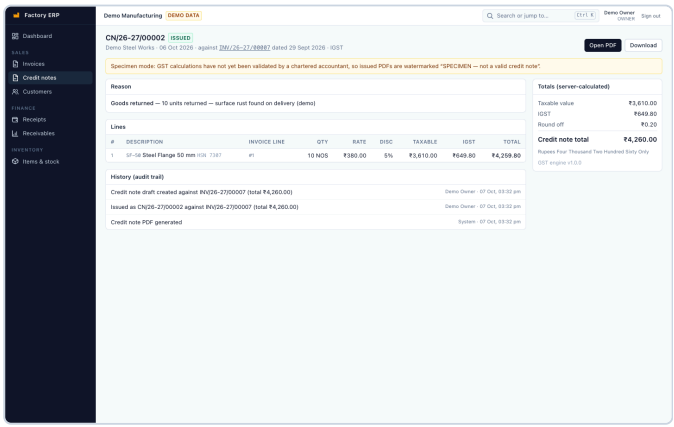
Every write that settles an invoice — allocating a payment, issuing a credit note, cancelling — first takes the same row lock on the invoice and reads paid and credited amounts *after* it. Deferred SQL checks (FO003/FO004) re-verify the invariant at commit. **Mutation-tested:** with the lock removed from credit-note issue, a forced-interleaving test fails because a credit note gets issued on stale figures; with the lock, it is rejected cleanly.

Your scenarios, as automated tests

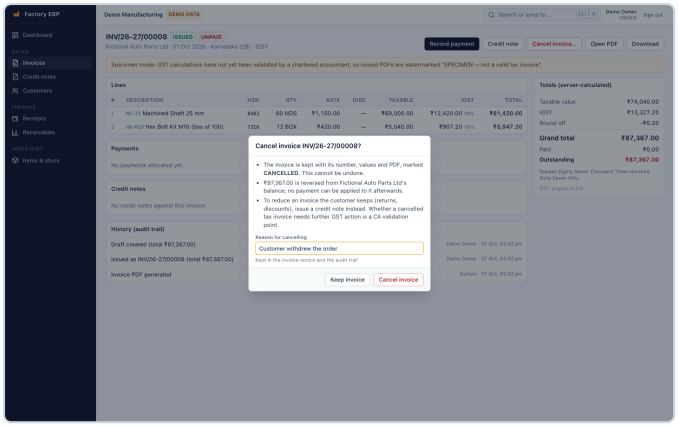
#	Scenario	Result
A	Invoice ₹10,000 → credit note ₹2,000	Outstanding ₹8,000 · Unpaid
B	Invoice ₹10,000 → receipt ₹6,000 → credit note ₹2,000	Outstanding ₹2,000 · Part-paid
C	Invoice ₹10,000 → credit note ₹10,000	Outstanding ₹0 · Credited
D	Cancel an unpaid invoice	Gone from open invoices, receivables, ageing and sales KPIs
E	Cancel twice	Rejected (`INVOICE_ALREADY_CANCELLED`)
F	Credit more than eligible	Rejected atomically — no number used, no ledger entry
G	Credit a different customer's invoice	Rejected (`CUSTOMER_MISMATCH`)
H	Push outstanding negative	Rejected — exactly the remainder is accepted
I	Receipts + allocations + credit notes together	Every derived balance reconciles; SQL view = read model



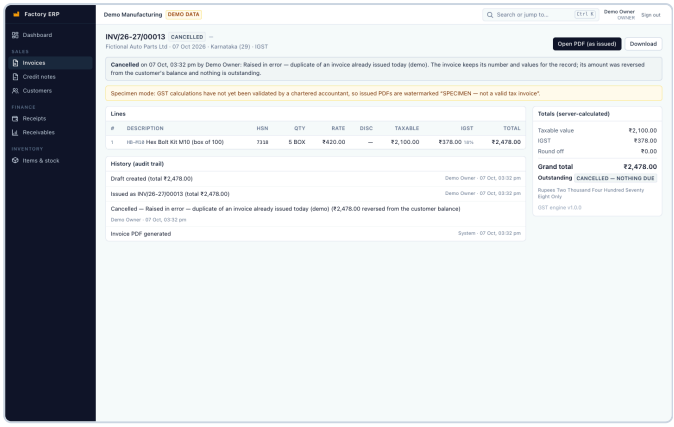
Credit-note editor: credit quantities and rates per invoice line, live engine preview, outstanding before and after



Credit note CN/26-27/00002: reason, lines, IGST, total credit, audit history, PDF



Cancel dialog: explicit consequences and a mandatory reason



Cancelled invoice: number and values kept, who/when/why shown, nothing due

Demo Manufacturing Pvt Ltd
Demo Manufacturing
Plot D-1, Demo Industrial Estate
Sample MIDC Area
Pune 411000
GSTIN: **27DEMOM0001A1Z1** · State: Maharashtra (27)
+91 00000 00000 · accounts@demo-manufacturing.example

CREDIT NOTE
Original for Recipient
DEMO DATA

CUSTOMER
Demo Steel Works
Survey 12, Demo GIDC Estate
Ahmedabad 380000
GSTIN: **24DEMOW0202B1Z3**
State: Gujarat (24)

CREDIT NOTE
Credit note no: **CN/26-27/00002**
Credit note date: **06-10-2026**
Against invoice: **INV/26-27/00007** dated 29-09-2026
Reason: Goods returned — 10 units returned — surface rust found on delivery (demo)
Place of supply: Gujarat (24)
Supply: Inter-state (IGST)

#	DESCRIPTION	HSN	QTY	RATE	DISC	TAXABLE	IGST	TOTAL
1	Steel Flange 50 mm	7307	10 NOS	380.00	5%	3,610.00	649.80 18%	4,259.80

GST RATE	TAXABLE	IGST
18%	3,610.00	649.80

Rupees Four Thousand Two Hundred Sixty Only

Taxable value	3,610.00
IGST	649.80
Round off	0.20
Total credit (₹)	**4,260.00**

For Demo Manufacturing Pvt Ltd

Authorised signatory

Computer-generated credit note · Factory ERP · GST engine v1.0.0 · SPECIMEN: tax calculations are pending CA validation and this document is not valid for tax purposes.

SPECIMEN — NOT A VALID CREDIT NO

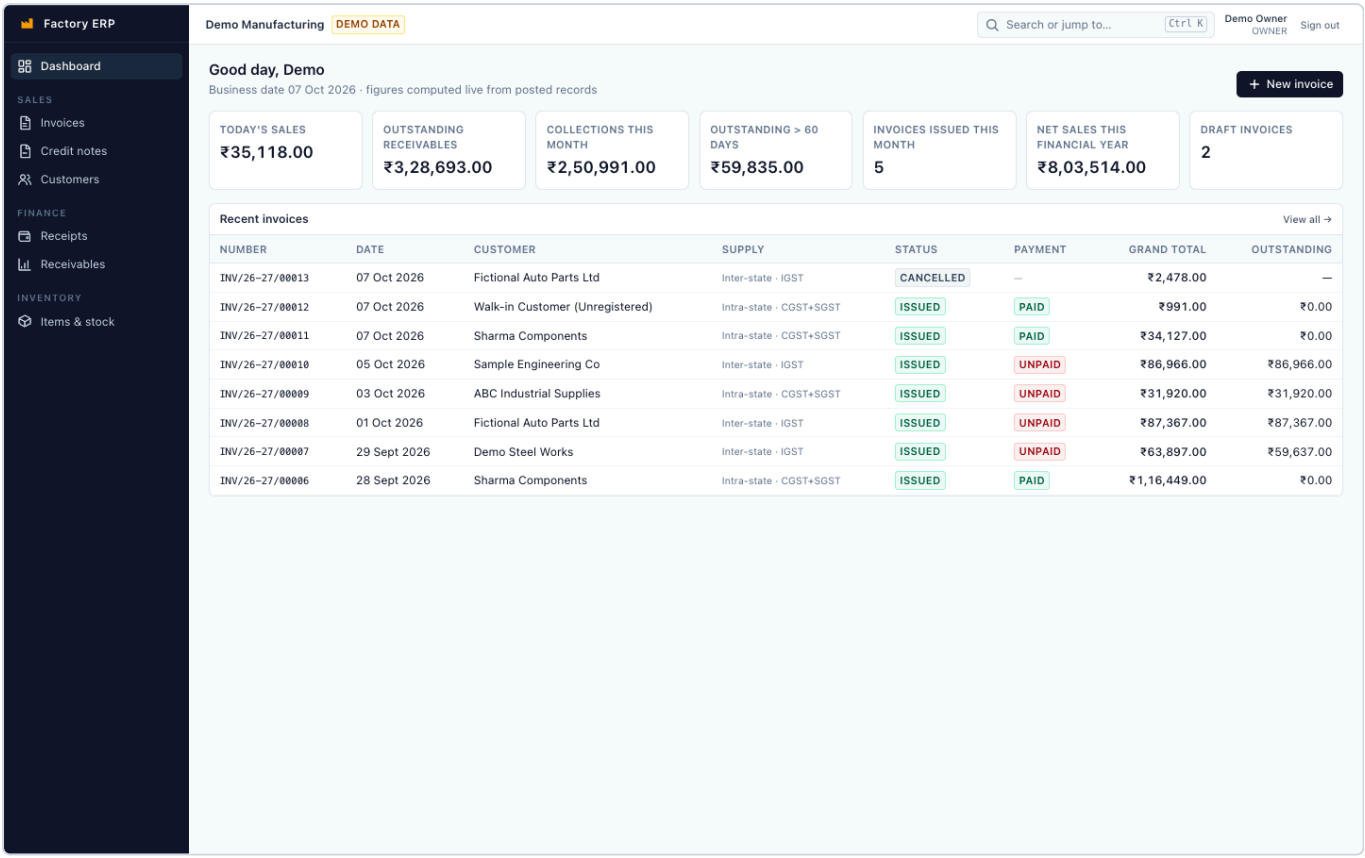
Credit-note PDF: CREDIT NOTE title, original invoice and reason, inherited 5% discount, IGST, SPECIMEN watermark, DEMO DATA badge

13 Dashboard & KPIs

Modules register KPIs in an **insights registry**; every KPI is backed by a real query and shown only to users holding its permission (a Worker sees none). There are no mock numbers.

KPI	Definition	Demo value
Today's sales	Invoices with today's date (IST), incl. GST; cancelled excluded; credit notes not deducted	₹35,118.00
Outstanding receivables	Customer ledger: invoices − cancellations − credit notes − receipts	₹3,28,693.00

Collections this month	Receipts dated in the current calendar month	₹2,50,991.00
Outstanding > 60 days	Outstanding on invoices older than 60 days, after payments and credit notes	₹59,835.00
Invoices issued this month	Count of issued invoices dated this month; cancelled excluded	5
Net sales this financial year	Invoices issued from 1 April minus credit notes issued in the year; cancelled excluded	₹8,03,514.00
Draft invoices	Invoices saved but not issued	2



Owner dashboard: 7 live KPIs; recent invoices with status (including CANCELLED), payment status and outstanding

14 API reference (37 endpoints, all under `/api/v1`)

Every command (POST/PUT/DELETE) requires an `Idempotency-Key` header and returns RFC 9457 problems on failure. Login and logout are handled outside the bus but are validated and audited.

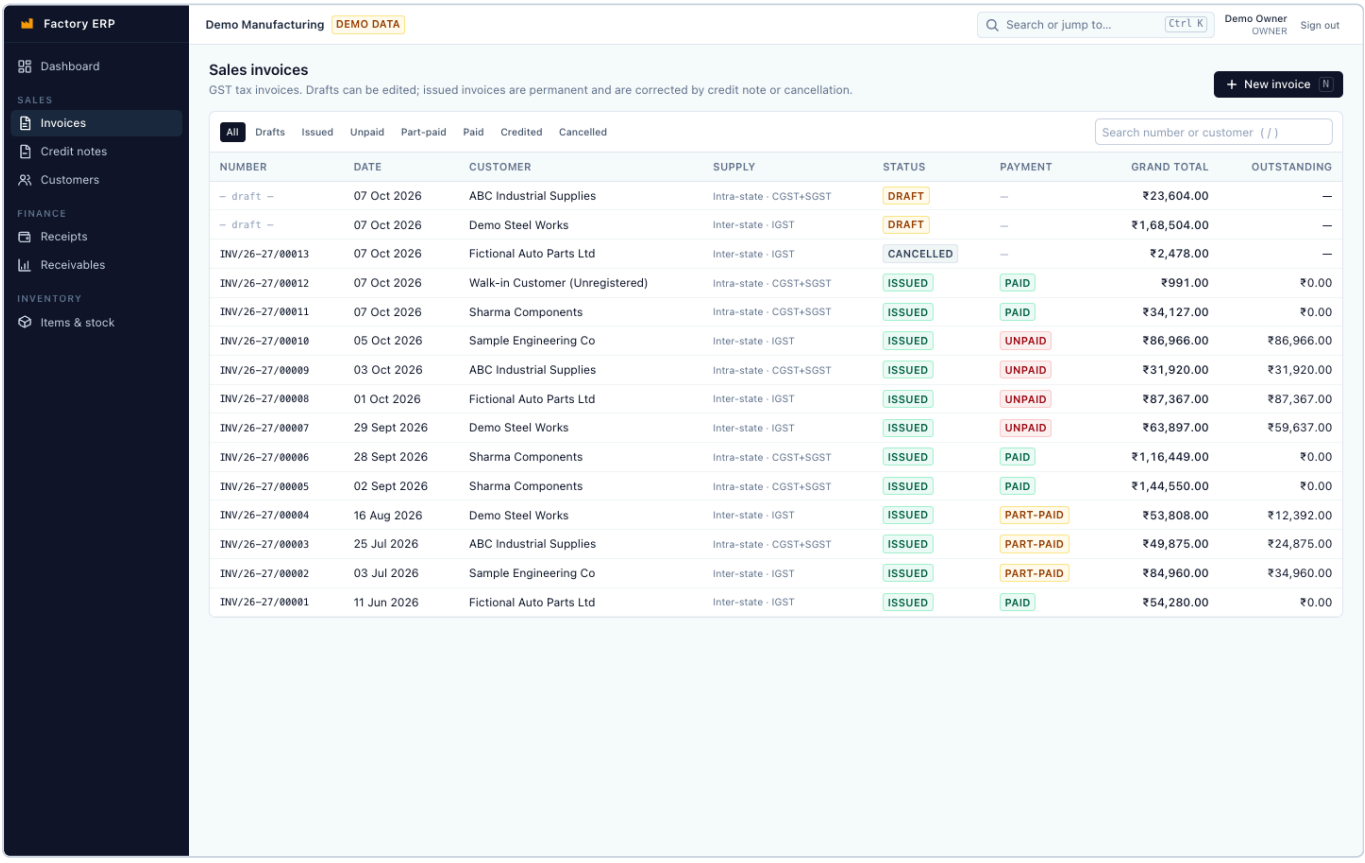
Method & path	Command / query	Permission
GET /health · GET /health/db	liveness · database latency	public
POST /auth/login · POST /auth/logout · GET /auth/me	session create · revoke · current user	public (rate-limited) · signed in
GET /company · /reference/states · /reference/hsn-codes · /reference/uoms	company and reference data	company.read · item.read
GET/POST /parties · GET /parties/:id	parties.list · create · get	party.read · party.create
GET/POST /items · GET /items/:id	items.list · create · get	item.read · item.create
GET /sales-invoices · GET /sales-invoices/:id	sales.invoices.list (status, payment status, customer, search) · get	sales.invoice.read
POST /sales-invoices · PUT · DELETE /sales-invoices/:id	createDraft · updateDraft · deleteDraft	sales.invoice.draft
POST /sales-invoices/:id/issue	sales.invoice.issue	sales.invoice.issue
POST /sales-invoices/:id/cancel new	sales.invoice.cancel	sales.invoice.cancel

POST /sales-invoices/:id/credit-notes new	sales.creditNote.createDraft	sales.credit_note.draft
GET /credit-notes · GET /credit-notes/:id new	sales.creditNotes.list · get	sales.invoice.read
PUT · DELETE /credit-notes/:id new	sales.creditNote.updateDraft · deleteDraft	sales.credit_note.draft
POST /credit-notes/:id/issue new	sales.creditNote.issue	sales.credit_note.issue
GET /receipts · GET /receipts/:id · POST /receipts · POST /receipts/:id/allocations	finance.receipts.list · get · receipt.record · receipt.allocate	finance.receivables.read · finance.receipt.record
GET /receivables/summary · GET /receivables/open-invoices	finance.receivables.summary · openInvoices	finance.receivables.read
GET /documents/:id/content	documents.content.get (hash-verified; AI-forbidden)	document.read
GET /audit-events · GET /dashboard	audit.events.list · dashboard.get	audit.read · company.read

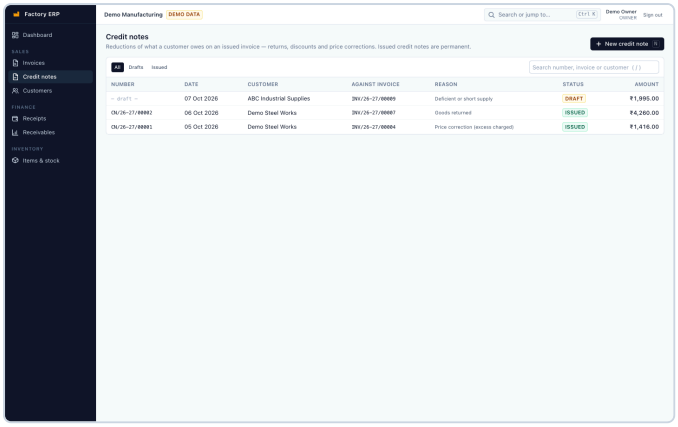
15 Web application

Area	Screens & features
Shell	Dark sidebar listing only working modules, filtered by permission: Dashboard; Sales (Invoices, **Credit notes**, Customers); Finance (Receipts, Receivables); Inventory (Items & stock). Top bar with company, DEMO DATA badge and user; mobile drawer.
Productivity	Ctrl/⌘+K command palette (actions plus live search across invoices, **credit notes**, customers and items); `/` to search; `N` for new documents; Ctrl/⌘+Enter to save.
Sales	Invoice list with All / Drafts / Issued / Unpaid / Part-paid / Paid / **Credited / Cancelled** tabs. Invoice detail with issue confirmation, PDF, payments, **credit notes, "Credit note" and "Cancel invoice…" actions**, cancellation banner and audit history. **Credit-note list, editor (with invoice picker) and detail with issue confirmation.**
Finance	Receipts list (unallocated filter), record receipt (prefill from an invoice), receipt detail, receivables ageing.
Quality	Indian number formatting and IST dates; problem+json errors shown inline; no horizontal page scroll on phones (390 px).

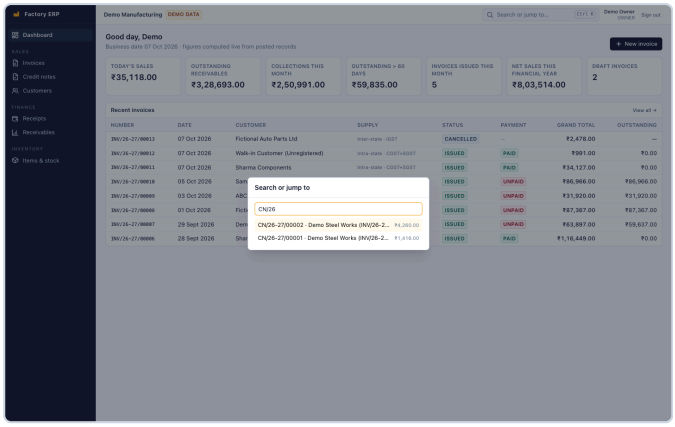
Screens (continued)



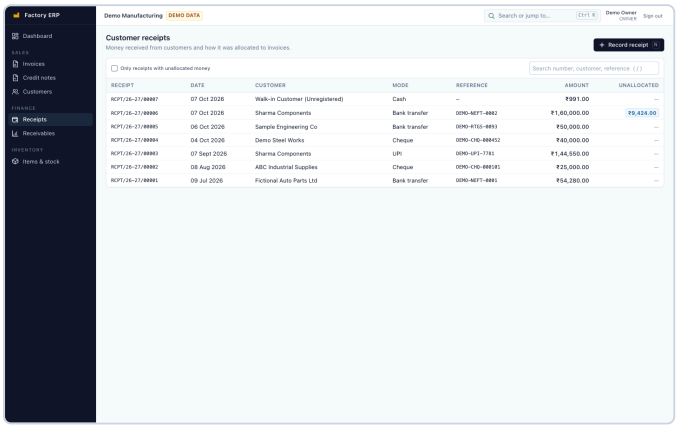
Invoice list: payment tabs, the new Credited and Cancelled tabs, CANCELLED badge in neutral grey



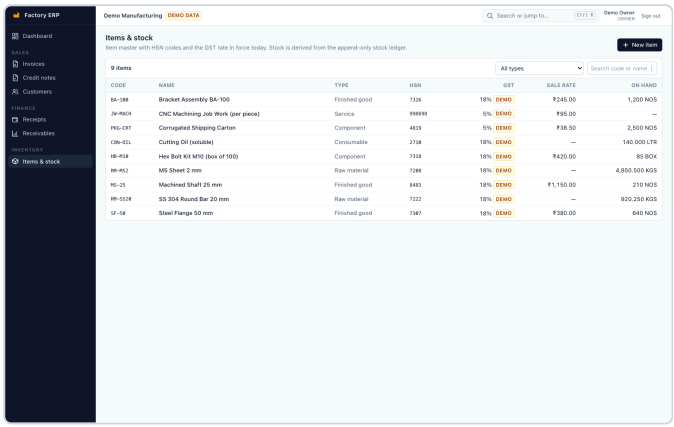
Credit-notes list: issued and draft credit notes with the invoice each one credits



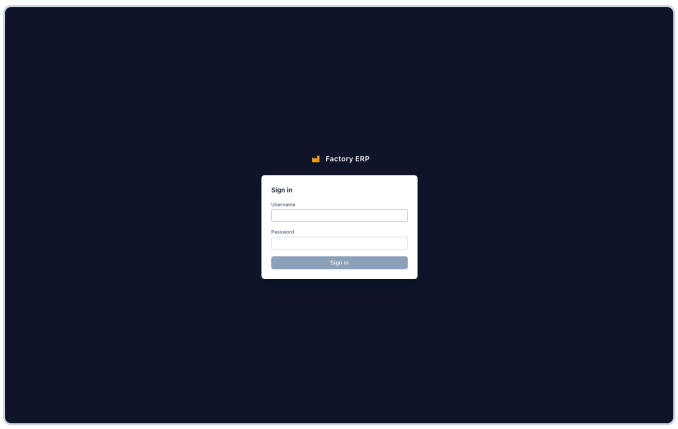
Command palette finding credit notes by number



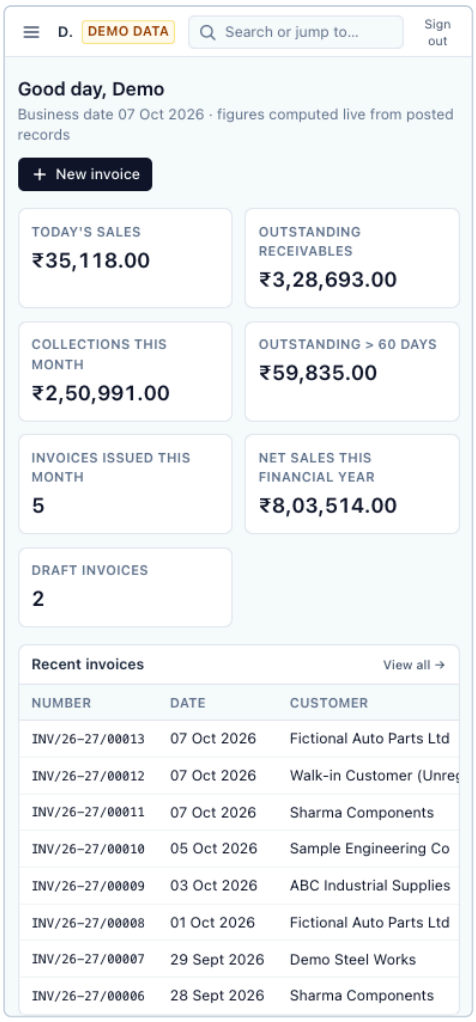
Receipts list: mode, reference, unallocated credit badge



Items & stock: HSN, GST rate in force today (demo-flagged), on-hand stock



Sign-in page



Mobile (390 px)

16 Demo data (synthetic, deterministic)

Everything is fictional. GSTINs use a PAN segment whose fourth character is "O" (not a valid PAN holder type), so they never collide with a real business; GST rates are tagged `DEMO_UNVERIFIED`; the seed refuses to run in production. All documents are created **through the same command bus as the UI**, so seeding exercises validation, audit, the ledger and the outbox.

Entity	Seeded
Company	Demo Manufacturing Pvt Ltd, Pune, Maharashtra (27). Finance readiness DEVELOPMENT, so PDFs are specimens.
Users (8)	owner, admin, accountant, production, purchase, hrmanager, supervisor, worker — password `factory-demo`.
Parties (9) · Items (9)	6 customers (intra- and inter-state, one unregistered) and 3 suppliers; finished goods, components, job work, raw materials and a consumable with opening stock.
Invoices (15)	13 issued (aged 0–118 days), of which **1 cancelled** (INV/26-27/00013, raised in error), plus 2 drafts. 13 invoice PDFs.
Receipts (7)	Full, part, cheque, UPI, cash, and one receipt paying two invoices with ₹9,424 left on account.
Credit notes (3)	**CN/26-27/00001** ₹1,416 — price correction on part-paid INV/26-27/00004 · **CN/26-27/00002** ₹4,260 — 10 units returned on INV/26-27/00007 (IGST, 5% discount inherited) · one draft of ₹1,995 (short supply) not yet issued. 2 credit-note PDFs.

Resulting receivables (all reconcile: balance = outstanding − unallocated = ledger)

Customer	Balance	Outstanding	Unallocated	0–30	31–60	61–90	90+
Sample Engineering Co	₹1,21,926.00	₹1,21,926.00	—	₹86,966.00	—	—	₹34,960.00
Fictional Auto Parts Ltd	₹87,367.00	₹87,367.00	—	₹87,367.00	—	—	—
Demo Steel Works	₹72,029.00	₹72,029.00	—	₹59,637.00	₹12,392.00	—	—
ABC Industrial Supplies	₹56,795.00	₹56,795.00	—	₹31,920.00	—	₹24,875.00	—
Sharma Components	−₹9,424.00	—	₹9,424.00	—	—	—	—
Total	**₹3,28,693.00**	**₹3,38,117.00**	**₹9,424.00**	**₹2,65,890.00**	**₹12,392.00**	**₹24,875.00**	**₹34,960.00**

Demo Steel Works dropped from ₹77,705 to ₹72,029 because of its two credit notes (₹1,416 + ₹4,260). Fictional Auto Parts' cancelled ₹2,478 invoice contributes nothing.

17 Testing & verification

Integration tests run against **real PostgreSQL**, never mocks. A global setup builds a migrated template database once per run (proving every migration applies to an empty database); each test file then clones its own isolated database.

Test file	Tests	Covers
core/sales/credit-notes.int.test.ts new	28	Drafts, scenarios A–C, F–I, IGST and discounts, cross-company isolation, gapless and concurrent numbering, financial-year rollover, racing credits, forced interleaving, FO003/FO004 at commit, immutability, idempotency, RBAC, AI tiers, audit, PDF via the outbox
core/sales/cancellation.int.test.ts new	14	Valid cancellation and ledger reversal, scenario D (receivables and KPI exclusion), E (twice), drafts/versions/reasons, paid and credited invoices, SQL-level immutability, payment-vs-cancel race, idempotency, RBAC, AI tier, audit
core/receivables.int.test.ts	21	Full, partial, multi-invoice and overpayment flows; atomic rejections; gapless receipts; concurrency incl. forced interleaving; deferred SQL check; ageing and reconciliation
core/invoices.int.test.ts	13	Server-side GST, IGST, versions, issue atomicity, date rules, gapless numbering under concurrency, immutability in SQL
core/api.int.test.ts	13	HTTP: health, login, security headers, Idempotency-Key, CSRF hardening, invoice flow + PDF, **credit-note and cancellation routes + real-Chrome credit-note PDF**
core/command-bus.int.test.ts	11	Validation, authentication, RBAC, audit, rollback, idempotency (incl. concurrent), AI tiers
core/static · migrate · roles (int)	14	Single-origin serving and SPA fallback; migration safety; reviewed role matrix snapshot
core/receivables/domain.test.ts	26	Allocation rules, overpayment, derived payment status incl. CREDITED and CANCELLED, ageing buckets
core/sales.test.ts · platform.test.ts	15	Number formatting, HTML escaping, invoice and **credit-note PDF templates**, argon2id; IST dates, UUIDv7, config
gst (engine, property, financial year)	22	Hand-worked cases, 400 random invoices per run, financial-year boundaries
money (values, format, words)	35	Exact arithmetic, rounding modes, Indian grouping, amount in words
web (preview, allocation-math, **credit-math**, format)	17	Client GST preview, oldest-first allocation, **credit-note preview and caps**, formatting
Total (22 files)	**229**	**All passing**

Mutation checks (proving the tests catch real failures)

- **Invoice numbering:** removing the series row lock makes the concurrency test fail with a duplicate number.
- **Payment allocation:** a forced-interleaving test fails without the invoice lock.
- **Credit-note issue:** with the settlement lock removed, the forced-interleaving test fails ("issued" on stale amounts); with the lock, the credit note is rejected cleanly. The lock was restored after the check.

Browser end-to-end journeys

One command builds a **throwaway database** with fresh demo data, starts the **production build** on its own port and data directory, runs every journey, fails on any server error log, then drops the database. The development database is never touched.

Smoke: 11/11

1. Signed-out redirect
2. Wrong password
3. Accountant signs in
4. Only working modules
5. Keyboard invoice
6. Issue with confirmation
7. PDF via the outbox
8. Palette search
9. Worker sees no sales
10. Mobile layout
11. No console errors

Receivables: 10/10

1. Sign in
2. Create customer
3. Issue ₹2,891 invoice
4. Partial payment ₹1,000
5. Part-paid, ₹1,891 left
6. Overpay ₹2,500
7. Paid in full
8. Customer in credit −₹609
9. Unallocated filter
10. No console errors

Corrections: 15/15

1. Sign in
2. J1 issue + cancel with reason
3. J1 cancelled state, nothing due
4. J1 no receivable remains
5. J2 partial credit note issued
6. J2 credit-note PDF
7. J3 receipt + credit → ₹1,313
8. J3 receivables agree
9. J4 full credit → ₹0, Credited
10. J5 editor blocks over-credit
11. J5 server rejects cleanly
12. J5 nothing changed, gapless
13. List + palette
14. Mobile pages
15. No console errors

Final verification results

Check	Result
Prettier format check · ESLint (type-aware) · strict typecheck of all 5 packages	**Pass**
Generated DB types match the schema (`--verify`)	**Pass**
Unit + property + integration tests	**229 / 229**
Production builds (core bundle, web build)	**Pass**
Browser E2E against the production build, fresh data	**36 / 36, zero server error logs**

18 Defects found & fixed during verification

Running the software, rather than only generating it, surfaced these issues. Each was fixed, and most now have a regression test.

Issue	Cause and fix
GST test expectations wrong (₹15.31 vs ₹15.32)	The hand calculation ignored per-component rounding. The test was corrected and the effect documented as a CA validation point.
Server crashed when serving the web app	Two not-found handlers were registered. Merged into one with SPA fallback; static-serving test added.
New JS bundle served as HTML after a rebuild	Static routes were fixed at startup. Files are resolved per request; a missing asset returns 404. Tests added.
Login lost the "return to" page	A global 401 handler raced the route guard. The guard now owns session redirects.
Invoice pages scrolled sideways on phones · quantity "100" shown as "1"	CSS grid `min-width` fix (checked in e2e); decimal-aware quantity helper with a test.
Race test not exercising the invoice lock	Found by mutation testing. A deterministic forced-interleaving test was added.
Server killed Chrome mid-render on shutdown new	Found by the new e2e runner's error-log check: Playwright's own SIGTERM handler closed the browser before the app's graceful shutdown. Disabled; in-flight PDFs now finish first.
Migration constraint name collisions new	Swapping a CHECK in one ALTER, and a constraint trigger named like an auto-named CHECK, collided. The migration rolled back cleanly (transactional); statements split and the trigger renamed.

Stale KPI description new	"Receipts are not recorded yet" was still shown; rewritten to describe invoices – cancellations – credit notes – receipts.
Unformatted amounts in rejection messages new	"₹1891.00" became "₹1,891.00" (Indian formatting) in credit-note rule messages.
Ambiguous e2e selectors new	"Credit note" and "Edit" also matched the "Credit notes" sidebar link (substring match). Switched to exact matching.

19 Running, building & CI

```
cp .env.example .env          # local settings (never commit real secrets)
pnpm install
pnpm db:up                     # PostgreSQL 18 in Docker (project "factory-erp"), bound to 127.0.0.1
pnpm db:seed                   # migrate + demo data + demo PDFs   (pnpm db:seed -- --reset rebuilds)
pnpm dev                       # API :3000 + web http://127.0.0.1:5173

pnpm verify                    # format + lint + typecheck + all tests
pnpm build                     # production bundles
pnpm --filter @factory-erp/web e2e     # all journeys on a throwaway database (dev DB untouched)
```

Demo sign-in: `accountant` / `factory-demo` covers invoicing, receipts, credit notes and cancellation; `worker` shows permission filtering.

CI (GitHub Actions on Ubuntu with a Postgres 18 service and Chrome): install (frozen lockfile) → format → lint → typecheck → migrate → generated-type drift check → all tests → build → e2e runner.

20 Documentation produced

Document	Content
`README.md`	What works, quick start, commands, layout, engineering rules, open CA points
`docs/ARCHITECTURE.md` (v0.4)	Master architecture, Round 2 review, regulatory register, CA validation pipeline; updated API sketch and AI tiers
`docs/adr/0001-foundation-stack-and-versions.md`	Verified versions and the reasons behind them
`docs/adr/0002-sales-correction-path.md` new	Cancellation and credit-note decisions, caps, concurrency, AI tiers, rename scope, CA validation points
`docs/discovery/`	Factory discovery material (deferred), renamed and re-rendered
This document	Development summary up to 7 October 2026

21 Limitations & open decisions

Decisions that need you (or your CA)

Decision	Context
Refunds, receipt reversal, un-allocation	Receipts and allocations are permanent. Until a policy is chosen, an invoice that has payments cannot be cancelled, and a credit note after full payment is rejected.
Cancelling issued tax invoices [VERIFY R-03/R-06]	Whether and until when an issued invoice may be cancelled rather than credited, and how it is reported. No time window is enforced yet.
Credit-note rules [VERIFY R-03]	Permissible grounds (our reason codes are a starting list), time limit, mandatory particulars, conditions for post-sale discounts.
GST on advances and on-account receipts	Excess receipts are held as customer credit; their tax treatment needs CA confirmation.

Rounding policy	Line-level HALF_UP and rupee rounding are engineering defaults. Several partial credit notes on one invoice can leave up to about ₹1 because each is rounded to the rupee.
Ageing basis · stock on returns	Ageing runs from the invoice date (due-date ageing needs a due date frozen at issue). Invoices and returns do not move stock in V1.
Remote-access provider (R2-D6)	Cloudflare Tunnel vs. alternatives.

Known limitations

- **Not valid for real tax documents yet.** GST rates are demo values; invoice and credit-note PDFs are watermarked SPECIMEN until finance readiness is PRODUCTION after CA validation.
- **Not built yet:** refunds and receipt reversals; debit notes; credit notes not linked to one invoice; e-invoice and e-way bill; Tally export; purchasing, production, workforce, maintenance and quality; full general ledger; document AI and the AI copilot.
- **A cancelled invoice's PDF** is not re-stamped; it is kept and labelled "as issued". Archived customers cannot receive credit notes.
- **PDF generation needs Chrome/Chromium** on the server. The web bundle is about 173 KB gzipped.

22 Recommended next steps

1. **Push the first commit to a private repository**, so CI runs on every change.
2. **CA review session:** rounding, credit-note rules, cancellation rules, advances — then decide the refund / un-allocation policy and build it.
3. **Tally export** (vouchers and masters as XML), so the CA keeps the official books (R2-D4).
4. **Due-date ageing and overdue alerts**, once the ageing basis is decided.
5. **Production packaging:** a Dockerfile for the factory server, a backup and restore drill, and remote access per R2-D6.
6. **Then the next modules** in the planned order: purchasing and inventory, shop floor, and the AI copilot on top of the existing command bus.

> **Bottom line**
>
> - Factory ERP has a solid, verified core: financial rules are enforced in both the application and the database, every change is audited, and concurrency is proven.
> - Three complete business flows work end to end: invoice to PDF, invoice to cash, and correcting an invoice by cancellation or credit note.
> - The AI copilot's guard rails are already in place: it will be able to prepare drafts, but never to issue, cancel or move money on its own.